E-Ayurvedic Health Assistant

System design, architecture and full project description

1. Project overview

E-Ayurvedic is a web application where a person can describe their symptoms, upload a medical report or photo (JPG, PNG or PDF), and instantly get general Ayurvedic and lifestyle guidance. The app uses the **Gemini API** to understand symptoms and to read uploaded files directly, so no separate disease dataset, OCR engine or PDF parser has to be built.

Users can create an account so that every past analysis is saved in a personal history. When symptoms look serious, the app shows an emergency alert and advises meeting a doctor immediately. The app gives guidance only and does not replace a doctor.

The original project idea (a PHP + MySQL + Android portal with doctors, payments and organ donation) has been simplified into a lean, modern web-only system that one developer can build and demo in a short time.

2. Objectives

- Give people quick, general health guidance at any time, without waiting in a queue.
- Suggest Ayurvedic herbs, fruits, diet and lifestyle tips matched to the symptoms.
- Understand uploaded lab reports, prescriptions and photos and explain them in simple language.
- Tell the user which type of specialist doctor to meet.
- Warn clearly in case of emergency symptoms.
- Save each analysis in a private history that only the user can see and delete.

3. Scope: what is included and what is removed

Included	Removed from the old design
Login (Google or email), symptoms form, file upload, Gemini analysis, result page, history page with delete	Android app, doctor module, admin panel, online payment, appointment booking, organ donor module, PHP/MySQL/XAMPP/AngularJS stack

4. Main features

- **Account and login:** Google sign-in or email and password, handled by Firebase Auth.
- **Symptoms form:** symptoms in free text, age, gender, duration of the problem.
- **File upload:** lab report, prescription, discharge summary or skin photo (JPG, PNG, PDF).
- **AI analysis:** possible conditions, abnormal report values, herbs and fruits with descriptions, diet, lifestyle, specialist suggestion, emergency flag.
- **History:** list of earlier analyses, open any one, delete one or all.
- **Disclaimer on every result:** this is not medical advice; confirm with a doctor.

5. System architecture

The system has three layers: a React frontend, a Node.js backend and external services (Firebase and Gemini). The browser never talks to Gemini directly, so the API key stays secret on the server.

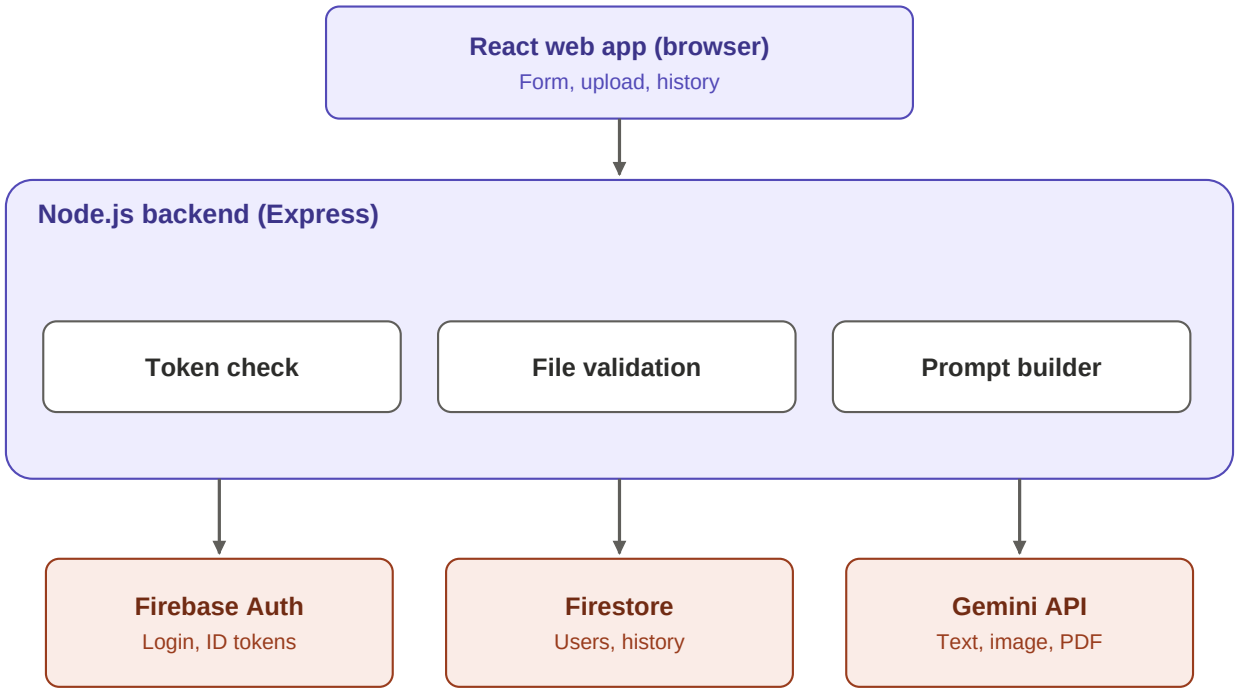

Figure 1: High-level system architecture

5.1 Layer 1: Frontend (React web app)

This is what the user sees. It contains the login page, the symptoms form with file upload, the result page and the history page. For login it talks directly to Firebase Auth and receives an ID token. This token is attached to every request sent to the backend.

5.2 Layer 2: Backend (Node.js + Express)

- **Token check:** verifies the Firebase ID token so only logged-in users can use the API.
- **File validation:** allows only JPG, PNG and PDF, enforces a size limit (for example 10 MB) and rejects everything else.
- **Prompt builder:** combines symptoms, age, gender and the file into one prompt with safety rules, and asks Gemini to answer in a fixed JSON format.
- The Gemini API key is stored only in a server environment variable.

5.3 Layer 3: External services

Service	Role
Firebase Auth	Handles sign-up, login and ID tokens. No password code needs to be written.
Firestore	Stores user profiles and the history of analyses. Security rules make sure each user sees only their own data.
Gemini API	Understands symptoms, reads images and PDFs directly, and returns structured guidance.

6. Request flow (one analysis)

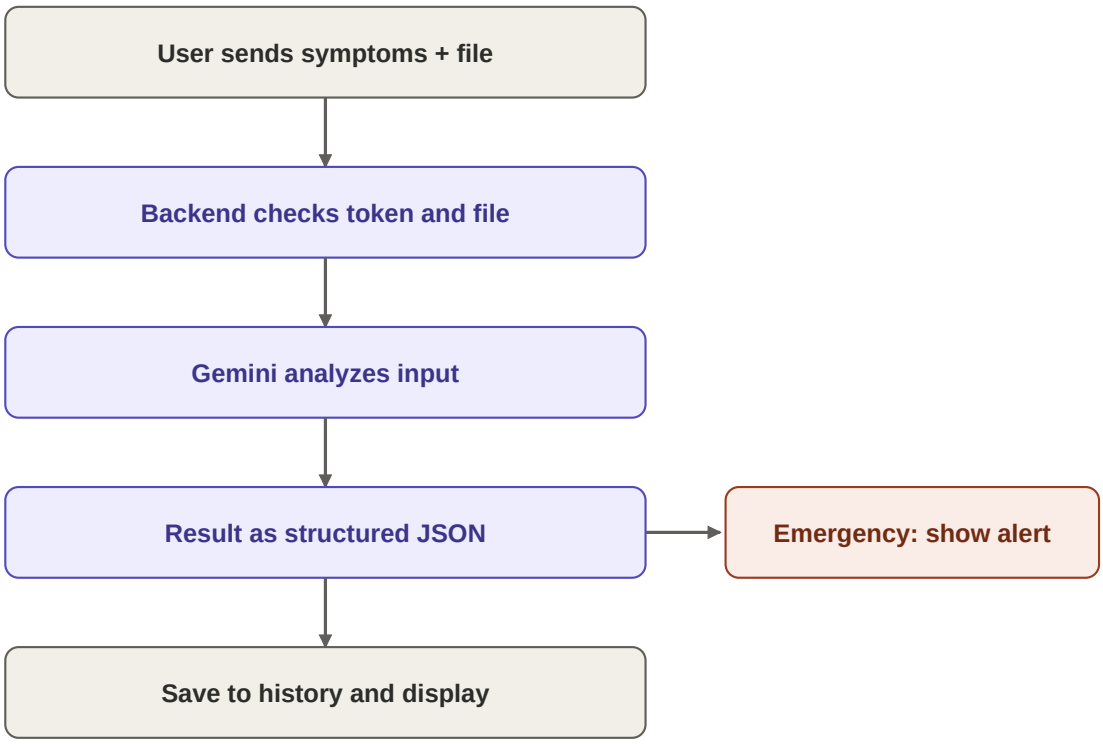

Figure 2: Flow of a single analysis request

- The user fills in the form and chooses a file.
- The backend checks the token and validates the file. If anything is wrong, the request stops here with an error message.
- The backend sends symptoms, age, gender and the file to Gemini. Name, email and phone are never sent.
- Gemini returns a fixed JSON structure (see section 8).
- If `emergency` is true, a red alert is shown first. The result is then saved in Firestore and shown as cards.

7. Database design (Firestore)

Only three collections are needed. One user has many history records, and one history record can have several files.

USERS

Field	Type	Meaning
uid (PK)	string	Firebase user id
name	string	Display name
email	string	Login email
created_at	timestamp	Account creation time

HISTORY

Field	Type	Meaning
id (PK)	string	Record id
user_uid (FK)	string	Owner of the record
symptoms	string	Text entered by the user
age, gender	int, string	Basic details used by the prompt
gemini_result	json	Full structured answer from Gemini
emergency	boolean	True if serious symptoms were flagged
created_at	timestamp	When the analysis was done

FILES

Field	Type	Meaning
id (PK)	string	File record id
history_id (FK)	string	Linked history record
file_type	string	jpg, png or pdf
file_name, size_kb	string, int	Basic metadata only

The original uploaded file is deleted after analysis. Only metadata and the result are kept, which reduces privacy risk.

8. Gemini integration

The backend sends one request containing the text prompt and, if present, the file. The prompt tells Gemini to give general guidance only, never a final diagnosis, to flag emergencies and to answer only in JSON. Small files (up to about 20 MB) can be sent directly in the request; larger PDFs can use the Gemini Files API. Gemini 2.x and later models can read images and PDFs natively.

Expected JSON response

```
{
  "conditions": ["Possible condition 1", "Possible condition 2"],
  "abnormal_values": [{"test": "Hemoglobin", "value": "9.8", "note": "Below normal"}],
  "herbs": [{"name": "Tulsi", "benefit": "Helps in cold and cough"}],
  "fruits_diet": ["Warm water", "Light khichdi"],
```

```
  "lifestyle": ["Sleep 7-8 hours", "Light walking"],
  "specialist": "General physician",
  "emergency": false,
  "disclaimer": "General guidance only. Please confirm with a doctor."
}
```

Prompt rules (summary)

- Act as a general wellness guide, not as a doctor. Do not give a final diagnosis or prescribe medicines.
- If symptoms suggest an emergency (chest pain, difficulty breathing, severe bleeding, stroke signs), set emergency to true.
- If a file is unclear or unreadable, say so instead of guessing.
- Return only valid JSON in the given structure.

9. Backend API endpoints

Endpoint	Purpose
POST /api/analyze	Receive symptoms and file, call Gemini, save the result to history, return the result
GET /api/history	List all past analyses of the logged-in user
GET /api/history/:id	Open one past analysis in detail
DELETE /api/history/:id	Delete one record
DELETE /api/history	Delete the full history of the user

Every endpoint requires a valid token, and a user can only read or delete their own records.

10. Frontend pages

Page	Content
Login / Sign up	Google sign-in or email and password
Home	Symptoms form, age, gender, duration, file upload with a tip to upload a clear, well-lit photo
Result	Emergency alert (if any), conditions, report values, herbs and fruits, diet, lifestyle, specialist, disclaimer
History	List of earlier analyses with open and delete options

11. Security and privacy

- Gemini API key lives only in a backend environment variable, never in frontend code.
- Firestore security rules allow access only when the record's user_uid equals the logged-in user's uid.
- Only symptoms, age, gender and the file are sent to Gemini. Name, email and phone are not sent.
- File type and size are validated; the uploaded file is deleted after analysis.
- Users can delete one record or their whole history at any time.
- Rate limiting on the backend prevents misuse of the AI endpoint.
- HTTPS is used everywhere (provided by the hosting platforms).
- Health data is sensitive, so a short privacy note should tell users that their file is processed by an AI service.

12. Safety and limitations

- The app gives general guidance. It must not be presented as a diagnosis or a replacement for a doctor.
- AI can make mistakes, especially with blurry photos or unusual reports, so results always carry a disclaimer.
- Ayurvedic suggestions are general and may not suit people with allergies, pregnancy, chronic conditions or those who take regular medicines. The app should advise consulting a doctor first in such cases.
- Skin or body photos give only a rough observation, never a confirmed result.
- Serious symptoms must always lead to an emergency suggestion, not home remedies.

13. Technology stack

Layer	Technology
Frontend	React + Bootstrap or Tailwind CSS
Backend	Node.js + Express + multer (file upload)
Login	Firebase Auth (Google or email)
Database	Firestore
AI	Gemini API (text, image and PDF input)
Hosting	Vercel (frontend), Render (backend), Firebase (auth and database)

14. Folder structure

```
ayurvedic-app/
|-- client/                 (React)
|   `-- src/
|       |-- pages/          Login, Home, Result, History
|       |-- components/     FormCard, ResultCard, EmergencyAlert
|       `-- firebase.js
`-- server/                 (Node + Express)
    |-- index.js
    |-- routes/             analyze.js, history.js
    |-- middleware/         auth.js, upload.js
    `-- services/           gemini.js, prompt.js
```

15. Development plan

Step	Work	Result
1	Set up React, Express and Firebase project	Empty app runs and login works
2	Build symptoms form and /api/analyze with Gemini (text only)	Text symptoms give structured result
3	Add file upload and validation	Photos and PDFs are analyzed
4	Save results, build history page and delete	Each user sees own history
5	Add emergency alert, disclaimer, Firestore rules, rate limit	Safe and demo-ready
6	Deploy and test on mobile browser	Public link to share

16. Future scope

- Hindi and English language toggle, and Hindi voice input.
- Nearby doctor search through a map link.
- Reminders for diet and herb routines.
- Doctor module, appointments and payment can be added later if needed.
- Progress charts for repeated lab values over time.

17. Conclusion

By using Gemini for understanding symptoms and reading files, and Firebase for login and storage, the old complex PHP portal becomes a small, modern and practical web app. It keeps the useful core of the original idea (Ayurvedic guidance and a personal health history) while adding file analysis, and it can be built, deployed and demonstrated in a short time.